MODERATED LEARNING MANAGEMENT SYSTEM

Five-Page Final Software Engineering Report | CS4433
National University of Lesotho • Individual Project • Release v1.1
Student: Lephahamela Matsoso | Student No.: 202321228

1. Project overview and engineering objectives

This individual project developed an offline desktop Learning Management System (LMS) for secondary-school assessment administration. The defining workflow separates lecturer marking from Head of Department (HoD) moderation and release, so students cannot view marks before approval. Four roles are supported: administrator, lecturer, HoD and student. The system covers user/course administration, enrolment, assignments, submissions, individual and class-wide extensions, marking, audit records and result publication.

The engineering goal was to build a maintainable, testable system with traceable requirements, modular architecture, controlled releases and measurable verification evidence—not merely a functioning graphical interface. The implementation uses C++17, Qt 6 Widgets, SQLite, CMake/Ninja, libsodium, GoogleTest and GitHub Actions.

2. Development process and project planning

A plan-driven Waterfall structure defined phase gates; V-Model thinking linked early artefacts with later verification. Requirements were reviewed before architectural design; domain rules were covered by unit tests, service–repository interactions by integration tests, and role workflows by GUI/acceptance checks. This was appropriate for a fixed academic brief and required documentation, but sequential phases risk late discovery of misunderstood requirements. Iterative corrections and regression testing therefore complemented the plan (Sommerville, 2016; Pressman and Maxim, 2020).

Figure 1. Project Gantt chart, dependencies and milestones. View diagram ↗️

3. Requirements engineering

Requirement quality and acceptance criteria: For the moderation requirement, an acceptance condition is that a student sees no mark while it is pending approval and sees the released result only after an authorised HoD action. For CR1, an acceptance condition is that the lecturer can grant a class-wide extension and an enrolled student sees the revised effective deadline. Expressing requirements as observable conditions reduces ambiguity and improves test design (ISO/IEC/IEEE, 2018).

Requirements engineering translated stakeholders' needs into an SRS with functional requirements (course administration, submissions, marking, moderation and extensions) and non-functional concerns (security, usability, reliability and maintainability). Each requirement should be clear, verifiable and traceable to design and tests; this follows the requirements-engineering guidance of ISO/IEC/IEEE 29148 (2018). Role-based use cases specify actors and system boundaries.

Use cases capture actor-system interactions and provide a basis for acceptance tests. The release rule was traced from the requirement through the moderation service and student view to automated tests. The class-wide extension change request (CR1) demonstrates how requirements evolve after the initial baseline.

Figure 2. System actors and use cases. View diagram ↗️

4. Software architecture and detailed design

Layered architecture separates Qt presentation, use-case application services, domain entities/interfaces and SQLite persistence. This improves cohesion because each layer addresses a distinct concern; it reduces coupling because business rules are not embedded in windows or SQL. The trade-off is more classes and interfaces, which increase navigation and initial development effort. The separation was justified by independent domain testing and the later extension change (Fowler, 2002; Sommerville, 2016).

Dependency inversion is concrete: AssignmentService refers to IAssignmentRepository and ICourseRepository, not directly to SQLite classes. Substituting fake implementations enables focused testing. SQLite was chosen for offline, device-local storage, simple deployment and low operational overhead. It is less suitable when numerous computers need concurrent direct access to one shared database; a client/server design would then be preferable (SQLite, n.d.-a).

Figure 3. Layered component architecture. View diagram ↗

4.1 Object-oriented design and patterns

The design applies SOLID selectively rather than treating each principle as a checklist. Single Responsibility: AssignmentService handles assignment use cases, while ModerationService manages moderation, so changes have narrower impact. Open/Closed: IPenaltyStrategy permits a different late-penalty implementation without rewriting callers. Liskov Substitution: repository implementations must honour their declared interface contracts, including error and lookup semantics. Interface Segregation: specialised assignment, course and submission interfaces avoid requiring clients to depend on unrelated persistence operations. Dependency Inversion: use-case services depend on domain interfaces, with SQLite adapters supplying concrete implementations. These decisions improve substitution, testability and maintenance, but additional abstractions have a cost (Fowler, 2002; Pressman and Maxim, 2020).

Figure 4. Domain class structure and interfaces. View diagram ↗

Figure 5. Application services and dependencies. View diagram ↗

Figure 6. SQLite repository/data-access classes. View diagram ↗

4.2 Behavioural modelling

The submission state model documents transitions around submission, marking, moderation and release. The activity model shows responsibilities across actors. Sequence diagrams trace calls during assignment submission and departmental mark approval, making interactions and the points where validation or authorisation occurs explicit.

Figure 7. Submission state machine. View diagram ↗

Figure 8. Activity diagram with swimlanes. View diagram ↗

Figure 9. Sequence: student submits an assignment. View diagram ↗

Figure 10. Sequence: HoD moderates and releases marks. View diagram ↗

5. Implementation, database and security

The codebase separates src/domain, src/application, src/data, src/app and src/ui. Domain classes such as User, Assignment, Submission and Extension validate rules and represent assessment state. Application services enforce permissions and coordinate operations; SQLite repositories persist users, course relationships, submissions, extension decisions, notifications and marking history.

The ERD models users, courses, enrolment, assignments, submissions, extensions and audit history using related tables and keys. Repository classes centralise SQL. SQLite transactions provide atomicity for coordinated updates, helping prevent partially applied class-wide changes (SQLite, n.d.-b). Local file storage avoids a separate database server, but deployment across multiple machines would require changes to concurrency, storage and access architecture.

Figure 11. Entity relationship diagram and key associations. View diagram ↗️

5.1 Business rules and access control

The late-submission policy accepts late work but deducts 10% of the assignment maximum per late day for up to three days; later work receives zero. Students may resubmit subject to the applicable deadline rules, and older versions remain recorded. Lecturers mark and provide feedback; HoDs moderate and release marks. Students see only results that have been released. These conditions must be enforced in application/domain logic, not solely by disabling interface controls.

Security measures include role-based access checks, Argon2id password hashing through libsodium and an audit trail for changes to marks (actor, timestamp, old/new values and reason). SQLite transaction support helps ensure multi-step data changes do not leave inconsistent partial state. Because it is an offline application, production operation also depends on file-system permissions, secure backups and trustworthy local deployment.

6. Requirements-to-design traceability

Traceability through change: Traceability supports both forward and backward reasoning: forward from a requirement to the services, database operations and tests that realise it, and backward from a change to the stakeholder need it serves. When CR1 changed the deadline rules, the impact assessment considered enrolment lookups, existing individual extensions, lecturer permissions and student display behaviour. This is more reliable than modifying only the visible interface.

- Administration/enrolment → AdminService + course repository → Admin and repository tests
- Assignments/extensions → AssignmentService + assignment repository → AssignmentService tests
- Submission/late penalty → SubmissionService + penalty strategy → Submission and penalty tests
- Moderation/release control → ModerationService + student view → Moderation/view tests
- CR1 class-wide extension → AssignmentService + LecturerWindow → Four new tests + manual check

This is a concise traceability overview. Detailed traceability and evidence remain in the phase documents and repository test files.

7. Verification, validation and quality assurance

Verification asks, ‘Are we building the product right?’—checking consistency with specified behaviour through unit, integration, repository and interface tests. Validation asks, ‘Are we building the right product?’—checking that stakeholders’ workflows are supported, such as a lecturer extending a class deadline and a student seeing the result. GoogleTest and CTest provided 489 passing automated tests (139.96 seconds); the lecturer/student manual exercise supplied limited validation evidence. Passing tests cannot prove the absence of defects (Sommerville, 2016; Pressman and Maxim, 2020).

A recorded gcovr baseline report for the domain and application filters showed 960 executed of 980 measured lines, or 98% line coverage. This report predates the final v1.1 code changes and must not be presented as v1.1-wide coverage. Lizard analysed 415 functions (6,150 non-comment source lines), reporting average cyclomatic complexity 2.3 and no functions above the configured complexity threshold of 15. cppcheck and clang-tidy were also used; their reports are retained under docs/evidence.

7.1 Test levels and interpretation

Testing levels address different risks: unit tests isolate penalty and state-transition rules; integration tests check service–repository collaboration; UI tests check selected role-window behaviour; regression testing checks that earlier functionality survives CR1. Static analysis and complexity metrics complement dynamic tests, but neither can replace review of important requirements and acceptance outcomes.

8. Configuration management and CI

Git provides version history, branching and configuration baselines; GitHub hosts the project and automated workflows. Feature branches isolate changes from main, while CI is configured to build and test on Windows and Linux. Tagged releases v1.0 and v1.1 identify the original and maintenance baselines. Commit messages and preserved test reports support reproducibility and review. Version v1.1 was merged to main and its GitHub release includes a packaged Windows ZIP.

9. Deployment and operational limits

Operational acceptance criteria: Before school use, a technician should verify first-run administrator setup, persistence after restart, role-specific authorisation, moderation before publication and restore from a tested backup. A clean-computer test should establish that Qt libraries and plugins are present. These are proposed deployment checks, not claims of tests already completed. Offline operation reduces network dependence but shifts backup, device access and recovery responsibilities to the institution.

The Windows distribution includes lms.exe, required runtime DLLs, Qt plugins, a demo launcher and manuals for the four roles. The application stores production data in a local SQLite database. The demo accounts and password are intended for evaluation only and must not be reused in production. The release archive was built and published, but portability on a separate Windows machine without the development environment was not independently verified. File paths to submitted work must remain accessible, and multi-machine concurrent access is outside the supported architecture.

Release URL: https://github.com/lephahamelamatsoso08-glitch/lms/releases/tag/v1.1

10. Software maintenance: change request CR1

Maintenance trade-offs and regression risk: Extending every currently enrolled student differs from changing the assignment's default deadline: new enrolments and pre-existing individual extensions require explicit policy choices. The v1.1 implementation uses the enrolment list and existing extension mechanism with coordinated transactions; the regression analysis therefore prioritised preserving later individual deadlines and rejecting inconsistent partial updates. Future work should test larger cohorts and additional enrolment changes.

CR1 required a lecturer to extend an assignment deadline for the whole class and display the amended deadlines in students' assignment lists. This was classified as perfective maintenance because it added new behaviour after the initial v1.0 release. Impact analysis identified lecturer UI, assignment services, enrolment lookups, extension persistence/transactions and potential regressions in individual extensions and deadline calculations.

Development proceeded on feature/class-wide-extensions. The implementation reused existing enrolment and assignment abstractions, introduced a lecturer action and included transactional coordination to avoid partial multi-student updates. Four new tests were added; all 489 automated tests passed, and manual acceptance testing confirmed the updated deadline was displayed. Changes were merged into main, documented in docs/maintenance/CR1_CLASS_WIDE_EXTENSIONS.md and published as tag/release v1.1. The manual check did not exhaustively test every combination of enrolment, submission status or failure.

11. Professional, legal and ethical considerations

Assessment data demands confidentiality, integrity, accountability and controlled access. The application uses role checks, Argon2id password hashing and mark-change auditing, while deployment guidance requires restricted backups and non-demo credentials. These are risk-reduction measures rather than a formal security certification. Secure development also requires threat review, dependency management and verification throughout the life cycle (Souppaya, Scarfone and Dodson, 2022).

12. Critical evaluation and conclusion

The project demonstrates a software engineering workflow spanning planning, SRS, UML design, layered C++ implementation, automated testing, source control, packaging and maintenance. Its strengths are explicit separation of concerns, testable business services, moderation/audit controls and evidence-based regression testing. Main limitations include single-instance local storage, file-path-based submission references and unverified portability on an independent Windows computer. Future work could introduce managed file storage, stronger backup/restore verification and a service-based architecture for concurrent school users.

References and repository evidence

The conceptual discussion is supported by independently published books, standards and primary technical documentation listed below; the LMS repository and its test logs are cited in the body only as project evidence, not as external academic authorities.

Source repository: GitHub LMS repository

Release: v1.1 Windows release

Academic and technical references (Harvard style)

Fowler, M. (2002) Patterns of Enterprise Application Architecture. Boston: Addison-Wesley. Available at: https://martinfowler.com/books/eaa.html (Accessed: 2 November 2026).

ISO/IEC/IEEE (2018) ISO/IEC/IEEE 29148:2018: Systems and software engineering—Life cycle processes—Requirements engineering. Geneva: ISO. Available at: https://www.iso.org/standard/72089.html (Accessed: 2 November 2026).

Pressman, R.S. and Maxim, B.R. (2020) Software Engineering: A Practitioner's Approach. 9th edn. New York: McGraw-Hill Education. Available at: https://www.mheducation.co.uk/ise-software-engineering-a-practitioner-s-approach-9781260548006-emea-group (Accessed: 2 November 2026).

Sommerville, I. (2016) Software Engineering. 10th edn. Harlow: Pearson. Available at: https://www.pearson.com/en-us/subject-catalog/p/software-engineering/P200000003258 (Accessed: 2 November 2026).

Souppaya, M., Scarfone, K. and Dodson, D. (2022) Secure Software Development Framework (SSDF) Version 1.1. NIST SP 800-218. Gaithersburg: NIST. Available at: https://doi.org/10.6028/NIST.SP.800-218 (Accessed: 2 November 2026).

SQLite (n.d.-a) Appropriate Uses For SQLite. Available at: https://www.sqlite.org/whentouse.html (Accessed: 2 November 2026).

SQLite (n.d.-b) SQLite Is Transactional. Available at: https://www.sqlite.org/transactional.html (Accessed: 2 November 2026).